编译概述

杨广亮

2026年3月

基本信息

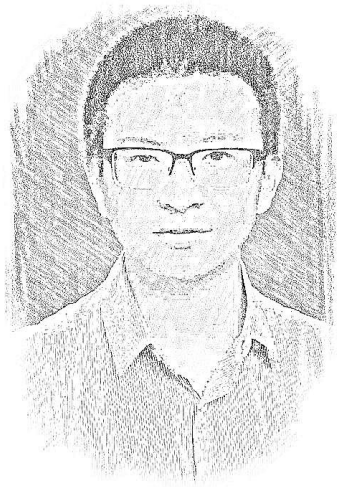
杨广亮：　复旦大学计算机学院 青年副研究员

研究领域：基础软件、系统安全和可靠性

办公室：　江湾二叉D6013

主页：　yian-lang.com/guangliang

邮箱：　yanggl at fudan.edu.cn

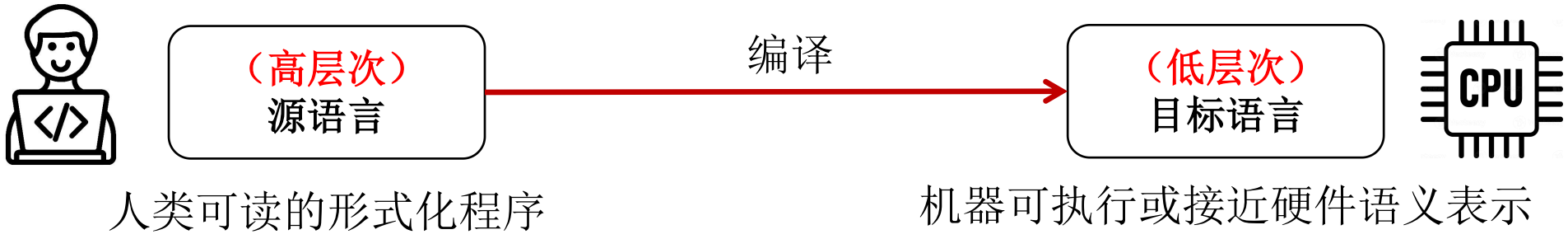


编译课程改革流程

- 23年：平行班（C语言编程）
- 24年：依托自研程序分析平台“莲花”开展教学，缩减实验（前端、中端）
- 25年：尝试引入AI系统相关内容
- 26年：引入自研编译器和系统语言“易安 yian”

什么是编译技术

➢ **编译**是指将高层次语言代码转换成低层次语言代码的过程



什么是编译技术

➢ **编译**是指将高层次语言代码转换成低层次语言代码的过程

人类可读的形式化程序

机器可执行或接近硬件语义表示

C/C++
Java
JavaScript
Python
Rust
Golang
PHP
易安
....

虚拟机器指令
汇编指令
中间语言**IR**
....

什么是编译技术

➢ **编译**是指将高层次语言代码转换成低层次语言代码的过程

人类可读的形式化程序

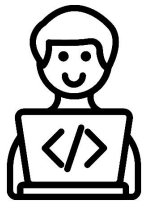
机器可执行或接近硬件语义表示

抽象
易书写
易阅读
易理解

具体
难以书写
难以阅读
难以理解

什么是AI编译

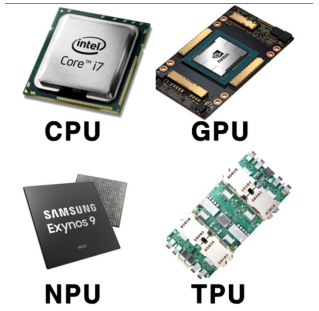
- AI**编译**是指将高层次AI模型转换成低层次执行表示的过程



（高层次）
AI模型

编译 →

（低层次）
目标代码



智能模型表达或任务语义

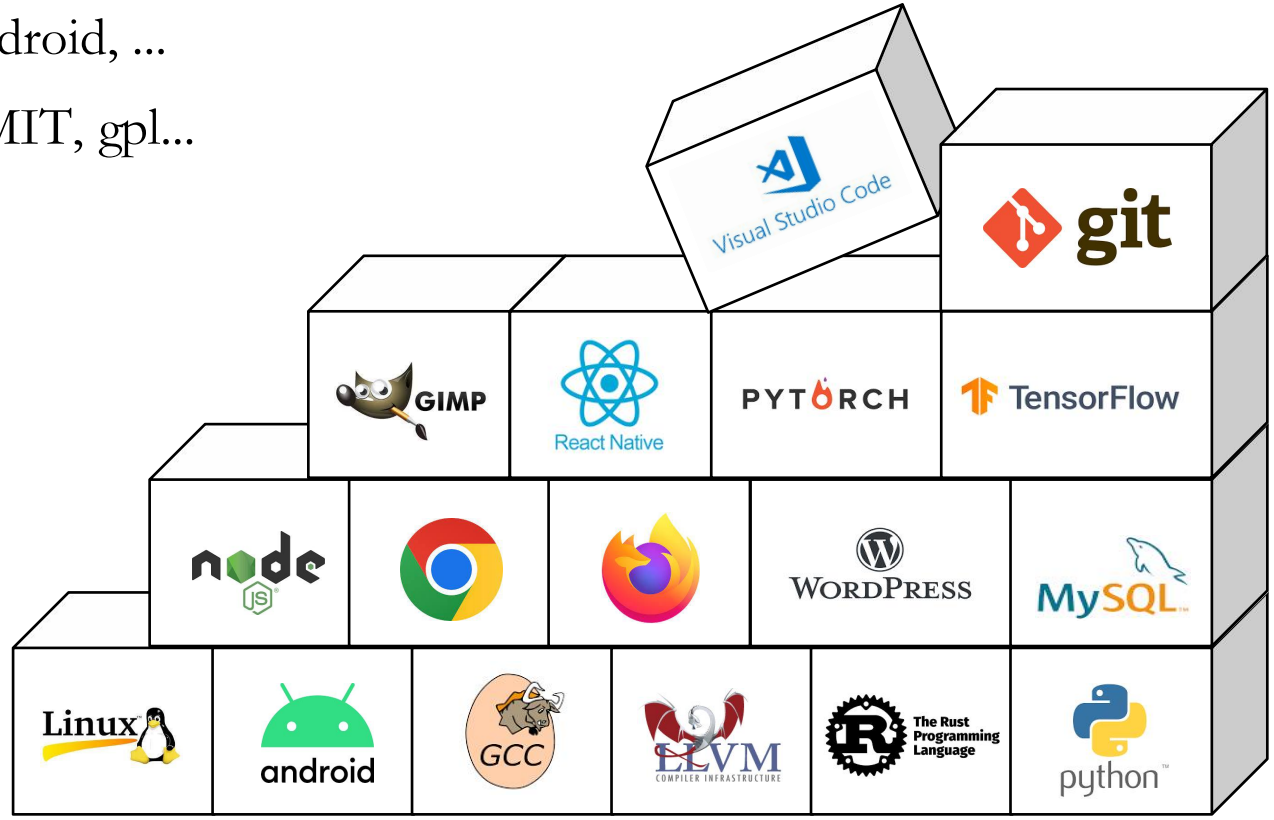
底层高效执行表示

神经网络计算图
张量算子
领域特定模型描述
自然语言任务约束
……

硬件指令序列
算子调度计划
加速器内核代码
优化后的中间表示
……

为什么学习这门课程

- 客观上，编译器在软件体系中起到了基石性作用
  - 随着开源运动发展，软件开发的方式发生了重大转变：通过组装开源代码快速形成可用软件
  - “共享代码”案例：linux, firefox, android, ...
  - 代码开发和分享协议：apache, bsd, MIT, gpl...



为什么学习这门课程

- 主观上，能够设计语言，编写自己的编译器
  - 理解计算机系统背后运行原理
  - 增加对代码工作原理了解
  - 提高代码书写质量
  - 代码优化
  - 发现和排除代码安全问题
  - 学习大型系统如何设计和实现

课程作业安排

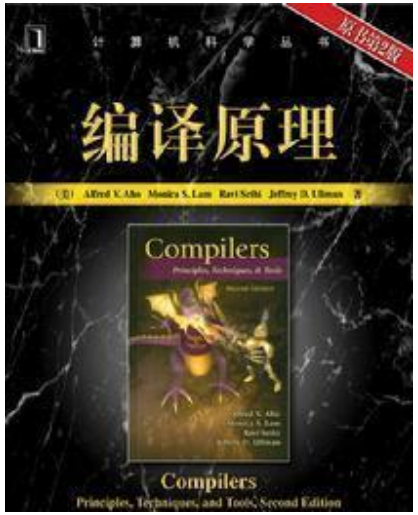
- 作业分组完成，由组长通过elearning提交：
  - 代码
  - 报告（包含代码思路、实验、组内贡献）
- 作业仓库： https://gitee.com/fdu-ssr/compiler2026spring

考核

- 课程作业：50%
  - 晚交作业：晚一天扣20%分数
- 考试：40%
- 出勤：10%
- 纪律要求和道德规范：
  - 严禁抄袭
  （包括大模型生成的文字、图片、文字、代码）

传统编译参考书

Alfred V. Aho 等《编译原理》

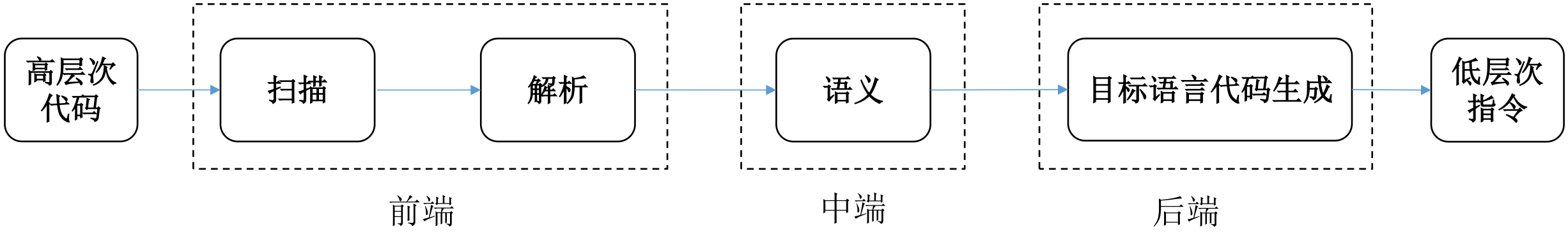


【补充】关于AI编译器材料：

- 《动手学深度学习》：https://zh.d2l.ai/
- 《机器学习系统：设计和实现》：https://openmlsys.github.io

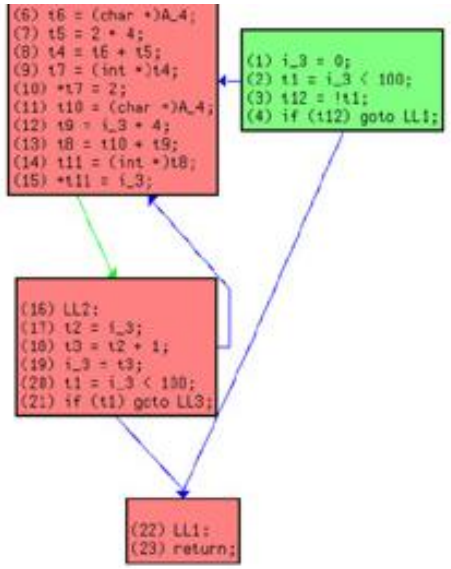
编译技术

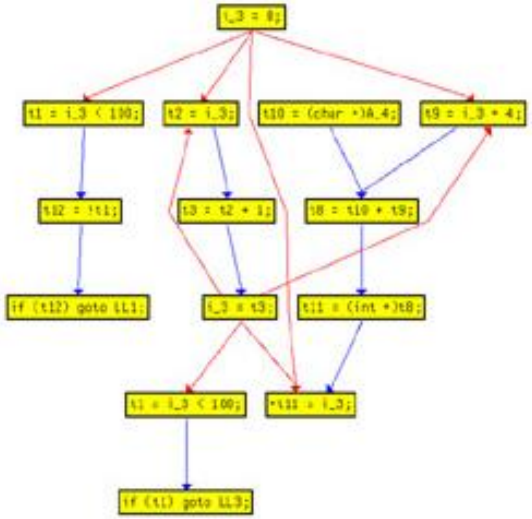
- **编译**是指将高层次语言代码转换成低层次语言代码的过程



扫描和解析

- 规范和分析语言格式
  - 数字，字符串
  - 条件分支，循环





优化

- 语义信息：什么含义
  - 函数调用关系
  - 成员变量包含关系
  - 数据传播关系
- 中间语言
  - LLVM
  - Jimple (soot)
  - GIR（General IR）
- 基于语义信息优化
  - 数据流
  - 控制流
  - 类型系统

GIR语法说明
https://yian-lang.net/lian/cn/03.frontend/3-2.gir/

ProgramUnits:
=ProgramUnit,file,path,package,import

Attributes:
=Built-in Types; Annotations; Custom Types

Locations:
=path,line,column

Declarations
= ClassDecl: <attribute> <name> <fields> <functions>
| VariableDecl:<attribute>,<name>
| ArraryDecl:<attribute><size><name>
| FunctionDecl:<attribute><name><args>

Expressions
=UnaryExpr:<operator><variable>
| BinaryExpr:<operator><v1><v2>
| FunctionCall:<function prototype><args>

Statements
=Assignment:<left><right>
|ContrlFlow:<condition><target>

代码生成和解释

- 将IR转化成目标语言
- 寄存器选择
- 指令优化
- 自上而下解释

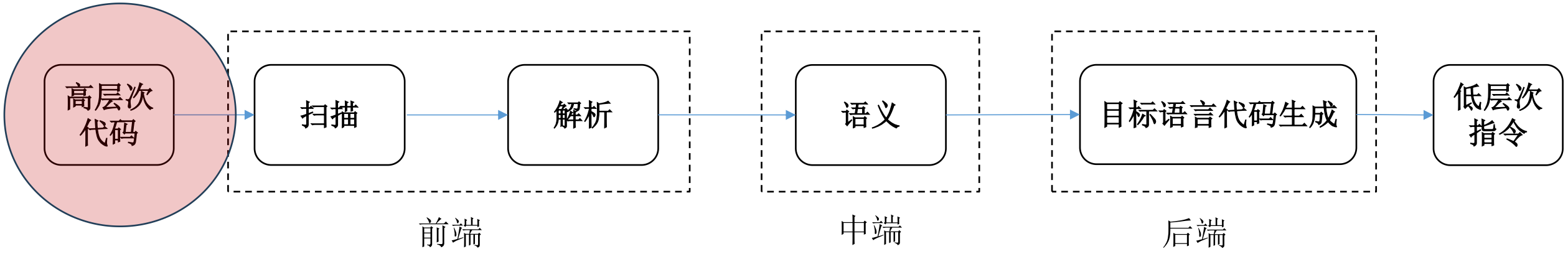
```
a = b + c

=>

1)  a.value = b.value + c.value
2)  a.type = b.type ^ c.type
3)  a.pos? b.pos? c.pos?
```

编译技术

➢ **编译**是指将高层次语言代码转换成低层次语言代码的过程



计算机语言

计算机 计算机科学 计算机专业

关注者 103 被浏览 104,320

计算机应该选择哪一种语言?

我现在已经大三了，日常学校教学数据结构之类课程都是C语言这些基础语言教学，然而我感觉在以后工作当中python之类的应用也许更加广泛，所以现在很痛苦，...显示全部

关注问题 写回答 邀请回答 好问题 15 2 条评论 分享 ···

编程语言 编程 计算机语言 值得买 计算机编程语言

关注者 85 被浏览 33,811

最值得学习的编程语言是哪个?

关注问题 写回答 邀请回答 好问题 1 条评论 分享 ···

Java Java 编程 Go 语言

关注者 28 被浏览 56,475

根据现在的局势，在未来5年内，JAVA和go哪个比较好，现在大三了，想确定一个方向?

关注问题 写回答 邀请回答 好问题 添加评论 分享 ···

C (Dennis Ritchie et al.)
1972-now

VM/GC/静态类型

Java (James Gosling @ Sun)
1995-now（JDK 19）

Php (Rasmus Lerdorf)
1994-now

JavaScript (Brendan Eich @ Netscape)
1995-now (ECMAScript 2021)

Rust (Graydon Hoare @ Mozilla)
2010 – now（1.64）

静态类型

TypeScript (MS)
2012（0.8）

C 语言基础

- 变量声明
  - 类型 变量名: int size;

类型

Type of the variable	Size	
	32-bit architecture	64-bit architecture
char	1	1
short int	2	2
int	4	≤8
long int	4	8
long long int	8	8
char *	4	8
float	4	4
double	8	8

变量名

[0-9 a-z A-Z _]

C 语言基础

- 基本运算
  - + - * / %
  - | & ~
  - || && !
  - =
  - ++ --
  - [] &

- 指针
  - &
  - + - ++ --

  ....

- 控制流
  - if else
  - switch case
  - for
  - while
  - break/continue

C 语言基础

- 函数声明
  - 类型 函数名（参数列表）
- 函数调用
  - 函数名（参数）
- 返回值
  - return xxx

```
long add(int x, int y)
{
        return x + y;
}

add(1, 2);
```

C 语言基础

- APIs 接口
  - 文件读写
  - 字符串操作
    - strlen, strcpy
  - 打印
    - printf

```
#include<stdio.h>
#include<string.h>

int main(int argc, char *argv[])
{
        // function body
        printf( “Hello world!\n” );

        return 0;
}
```

Java基础

- 变量声明
  - 类型 变量名: int i;

- 基本运算
  - + - * / %
  - | & ~
  - || && !
  - =
  - []

- 控制流
  - if else
  - switch case
  - for
  - while
  - break/continue

Java基础

```
class <name> extends <parent> implements <interface1,2,···> {
    <type> <name>;
    <type> <name>(<parameters>) {
        statements
    }
}
```

Rust基础

```
// Safety is the main focus

let [mut] <variable>: <type> = <value>

fn <函数名> -> <return value> ( <参数列表> )
        <函数体>
```

Rust基础

```
struct Rectangle {
    width: u32,
    height: u32,
}

impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }
    fn wider(&self, rect: &Rectangle) -> bool {
        self.width > rect.width
    }
}

fn main() {
    let rect1 = Rectangle { width: 30, height: 50 };
    let rect2 = Rectangle { width: 40, height: 20 };

    println!("{}", rect1.wider(&rect2));
}
```

- 基本运算
  - + - * / %
  - | & ~
  - || && !
  - =
  - [] &

- 控制流
  - if else
  - match
  - for
  - while
  - break/continue

Ownership和生命周期

- 黄金法则
  - Each value in Rust has an owner.
  - There can only be one owner at a time.
  - When the owner goes out of scope, the value will be dropped.

=> owner + scope + value

Ownership和生命周期

- Ownership

```
let s1 = String::from("hello");
let s2 = s1;                    // error

println!("{s1}, world!");
```

```
$ cargo run
   Compiling ownership v0.1.0 (file:///projects/ownership)
error[E0382]: borrow of moved value: `s1`
 --> src/main.rs:5:15
  |
2 |     let s1 = String::from("hello");
  |         -- move occurs because `s1` has type `String`, which does not implement the `Copy` trait
3 |     let s2 = s1;
  |              -- value moved here
4 |
5 |     println!("{s1}, world!");
  |               ^^^^ value borrowed here after move
  |
  = note: this error originates in the macro `$crate::format_args_nl` which comes from the expansion of the
macro `println` (in Nightly builds, run with -Z macro-backtrace for more info)
help: consider cloning the value if the performance cost is acceptable
  |
3 |     let s2 = s1.clone();
  |               ++++++++

For more information about this error, try `rustc --explain E0382`.
error: could not compile `ownership` (bin "ownership") due to 1 previous error
```

Ownership和生命周期

- Ownership

```
let s1 = String::from("hello");
let s2 = s1;                    // error

println!("{s1}, world!");
```

```
let mut s = String::from("hello");

let r1 = &s;
let r2 = &s;
let r3 = &mut s;          // error

println!("{}, {}, and {}", r1, r2, r3);
```

Ownership和生命周期

- Ownership

```
let s1 = String::from("hello");
let s2 = s1;                    // error

println!("{s1}, world!");
```

```
let mut s = String::from("hello");

let r1 = &s;
let r2 = &s;
let r3 = &mut s;            // error

println!("{}, {}, and {}", r1, r2, r3);
```

```
let mut x = 10;
let r1 = &mut x;
let r2 = &x;                    // error
```

Ownership和生命周期

- 黄金法则
  - Each value in Rust has an owner.
  - There can only be one owner at a time.
  - When the owner goes out of scope, the value will be dropped.

=> owner + scope + value

Ownership和生命周期

- Scope生命周期 ’letter

全局

```
let s = "hello";
```

限制

```
{                          // s is not valid here, it’ s not yet declared
    let s = "hello";       // s is valid from this point forward

                           // do stuff with s
}                          // this scope is now over, and s is no longer valid
```

Ownership和生命周期

基础引用 &T
（借数据，但不准改）

```
let s = "hello";
let ref_s: &str = &s;
```

Ownership和生命周期

基础引用 &T
（借数据，但不准改）

```
let s = "hello";
let ref_s: &str = &s;
```

```
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}
```

missing lifetime specifier: 返回值需要生命周期注解
帮助：试着改成 -> &'a str...

Ownership和生命周期

基础引用 &T
（借数据，但不准改）

带生命周期标注的引用 &'a T
（借数据，但必须保证在规定时间'a内归还）

```
let s = "hello";
let ref_s: &str = &s;
```

```
fn find<'a>(data: &'a [i32]) -> &'a i32
```

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}
```

Ownership和生命周期

基础引用 &T
（借数据，但不准改）

```
let s = "hello";
let ref_s: &str = &s;
```

带生命周期标注的引用 &'a T
（借数据，但必须保证在规定时间'a内归还）

```
fn find<'a>(data: &'a [i32]) -> &'a i32
```

可变借用 &'a mut T
（借数据，还可以修改）

```
let mut s = String::new();
let ref = &mut s;
```

Ownership和生命周期

基础引用 &T
（借你数据，但不准改）

```
let s = "hello";
let ref_s: &str = &s;
```

带生命周期标注的引用 &'a T
（借数据，但必须保证在规定时间'a内归还）

```
fn find<'a>(data: &'a [i32]) -> &'a i32
```

可变借用 &'a mut T
（借数据，还可以修改）

```
let mut s = String::new();
let ref = &mut s;
```

```
fn main() {
    let s = String::from("hello");

    change(&s);
}

fn change(some_string: &String) {
    some_string.push_str(", world");
}
```

```
$ cargo run
   Compiling ownership v0.1.0 (file:///projects/ownership)
error[E0596]: cannot borrow `*some_string` as mutable, as it is behind a `&` reference
 --> src/main.rs:8:5
  |
8 |     some_string.push_str(", world");
  |     ^^^^^^^^^^^^ `some_string` is a `&` reference, so the data it refers to cannot
be borrowed as mutable
  |
help: consider changing this to be a mutable reference
  |
7 | fn change(some_string: &mut String) {
  |                        +++

For more information about this error, try `rustc --explain E0596`.
error: could not compile `ownership` (bin "ownership") due to 1 previous error
```

Ownership和生命周期

基础引用 &T
（借你数据，但不准改）

```
let s = "hello";
let ref_s: &str = &s;
```

带生命周期标注的引用 &'a T
（借数据，但必须保证在规定时间'a内归还）

```
fn find<'a>(data: &'a [i32]) -> &'a i32
```

可变借用 &'a mut T
（借数据，还可以修改）

```
let mut s = String::new();
let ref = &mut s;
```

```
fn main() {
    let s = String::from("hello");

    change(&s);
}

fn change(some_string: &String) {
    some_string.push_str(", world");
}
```

```
fn main() {
    let mut s = String::from("hello");

    change(&mut s);
}

fn change(some_string: &mut String) {
    some_string.push_str(", world");
}
```

Ownership和生命周期

基础引用 &T
（借你数据，但不准改）

```
let s = "hello";
let ref_s: &str = &s;
```

带生命周期标注的引用 &'a T
（借数据，但必须保证在规定时间'a内归还）

```
fn find<'a>(data: &'a [i32]) -> &'a i32
```

可变借用 &'a mut T
（借数据，还可以修改）

```
let mut s = String::new();
let ref = &mut s;
```

```
fn update_score<'a>(score: &'a i32, delta:
i32) {
    *score += delta; // 通过可变引用修改值
    println!("新分数: {}", score);
}
```

```
fn update_score<'a>(score: &'a mut i32,
delta: i32) {
    *score += delta; // 通过可变引用修改值
    println!("新分数: {}", score);
}
```

Ownership和生命周期

- 真实代码

```
pub(crate) type NodeTaskHandlerClosure = Box<
    dyn for<'a, 'b, 'c> Fn(
        Box<dyn AsyncTaskResult>,
        &mut dyn ScriptTrait,
        &mut ScriptContext<'a, 'b, 'c>,
    ),
>;
pub(crate) type PluginTaskHandler = Box<
    dyn for<'a, 'b> Fn(
        Box<dyn AsyncTaskResult>,
        &'a mut [PluginContainer],
        &mut PluginContext<'a, 'b>,
    ),
>;
struct SourceCollector<'a, 'tcx> {
    cx: &'a mut Context<'tcx>,
    dst: PathBuf,
    emitted_local_sources: FxHashSet<PathBuf>,
    crate_name: &'a str,
}
impl DocVisitor<'_> for SourceCollector<'_, '_> {
```

Php语言

- According to the latest metrics from W3Techs, in 2024, PHP still powers 76.5% of websites where the server-side programming language can be discerned.

```
<!DOCTYPE html>
<html>
<body>

<h1>My first PHP page</h1>

<?php
echo "Hello World!";
?>

</body>
</html>
```

Php语言

- 变量命名：$<name>

```
$txt = "Fudan";
echo "I love $txt!";
```

- 类型
  - Integer
  - Float
  - Number
  - Strings
  - Infinity
  - NaN

- 基本运算
  - + - * / % ,++,--
  - | & ~
  - and or && ||
  - =, ==
  - []

- 控制流
  - if elif else
  - switch case
  - for/ for in/ foreach
  - while/ do while
  - break/continue

Php语言

- Array
  - $myArr = array("Volvo", 15, ["apples", "bananas"], myFunction);
- 函数调用

```
function sum($x, $y) {
  $z = $x + $y;
  return $z;
}
```

Php语言

```
class Fruit {
  // Properties
  public $name;
  public $color;

  // Methods
  function set_name($name) {
    $this->name = $name;
  }
  function get_name() {
    return $this->name;
  }
}
```

JavaScript基础

- Numbers（数字）
- String（字符串）
- List（列表）
- Tuple（元组）
- Object（字典）

声明：var/let <variable> = <value>

理念：Everything is an object

- 基本运算
  - + - * / %
  - | & ~
  - || && !
  - =
  - ++ --
  - []

- 控制流
  - if else
  - switch case
  - for
  - while
  - break/continue

JavaScript基础

```
var person={
     xing   : “Yang",
     ming   : “Guangliang",
     id     :  1234
     fullName : function() {
       return this.xing + " " + this.ming;
    }};


person.ming, person[“ming”]
```

TypeScript

- 变量声明
  - var/let name: type = value
- 类型
  - number：数字
  - string：字符串 （单引号和双引号都可以）
  - boolean：布尔型
  - 数组声明
    - let a: number[] = [1, 2];
  - enum枚举
    - enum Color {Red, Green, Blue}; enum Color {Red, Green, Blue};
  - Tuple元组
    - let x: [string, number]; x = ['Runoob', 1];

TypeScript

- 基本运算
  - + - * / %
  - | & ~
  - || && !
  - =
  - ++ --
  - []

- 控制流
  - if else
  - switch case
  - for
  - while
  - break/continue

TypeScript

function <name>(param1 [:datatype], param2 [:datatype]) : <return_type> {

    // 语句

}

<name>();

例子：

```
function add(x: number, y: number): number {
    return x + y;
}
add(1,2)
```

TypeScript

```
class <name> {
    // 成员变量
    var_name : type_name;
    // 构造函数
    <name>(p1:type1) {
        this.xxx = p1
    }
    display():void {
        console.log(this.xxx)
    }
}
```

主要编程范式的诞生：可计算性问题

大卫.希尔伯特
（数学的完备性问题）
1900年

- 完备性：所有数学命题都可以用一组有限的公理证明或证否；
- 一致性：可以证明的都是真命题。

库尔特.哥德尔
（不完备定理）
1931年

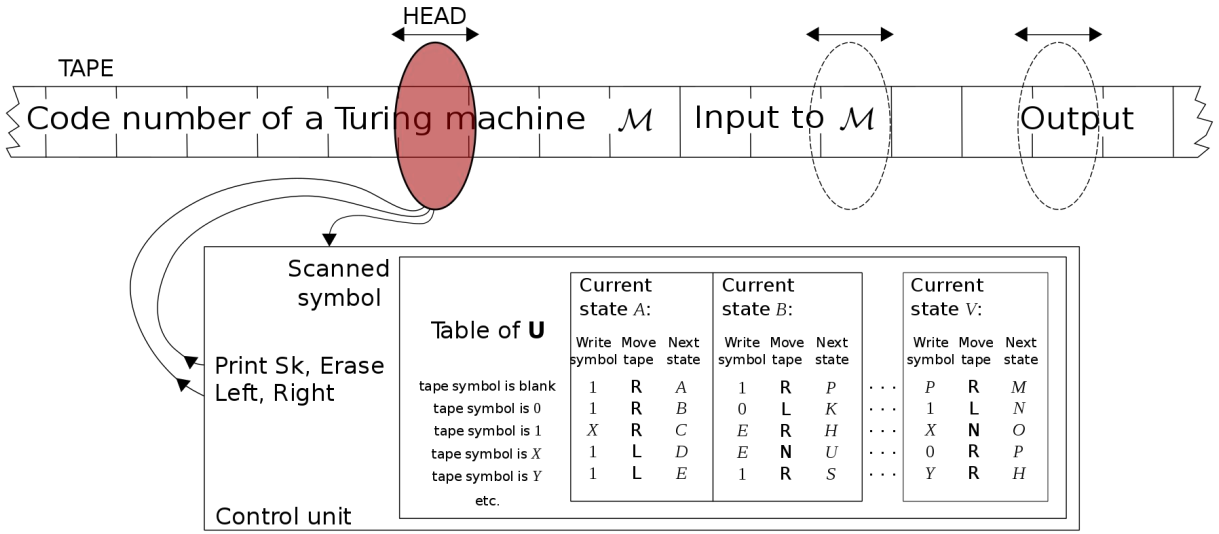
- 反例：“这个命题是不可证的”
- 如果完备性成立，则一致性不成立；
- 如果一致性成立，则完备性不成立。

邱奇和图灵
（可计算性）
1936年

- 可判定性：哪些问题是在有限时间内可以计算的？
- 邱奇：可计算函数和Lambda演绎
- 可计算函数可以等价为图灵机能计算的函数

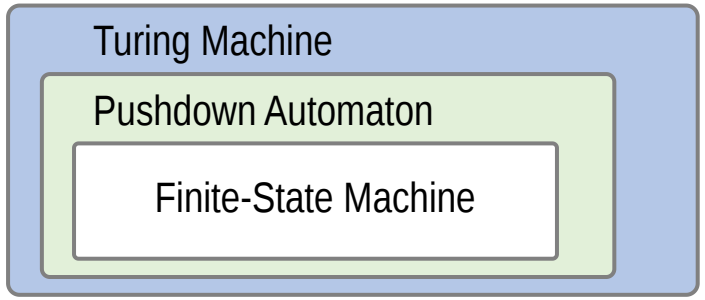
图灵机启发命令式编程

- 通用图灵机：
  - 一条无限长的纸带TAPE
  - 一个读写头HEAD
  - 一个状态寄存器
  - 一套控制规则TABLE：根据当前的寄存器值和读写头所指的格子上的符号确定下一步的动作



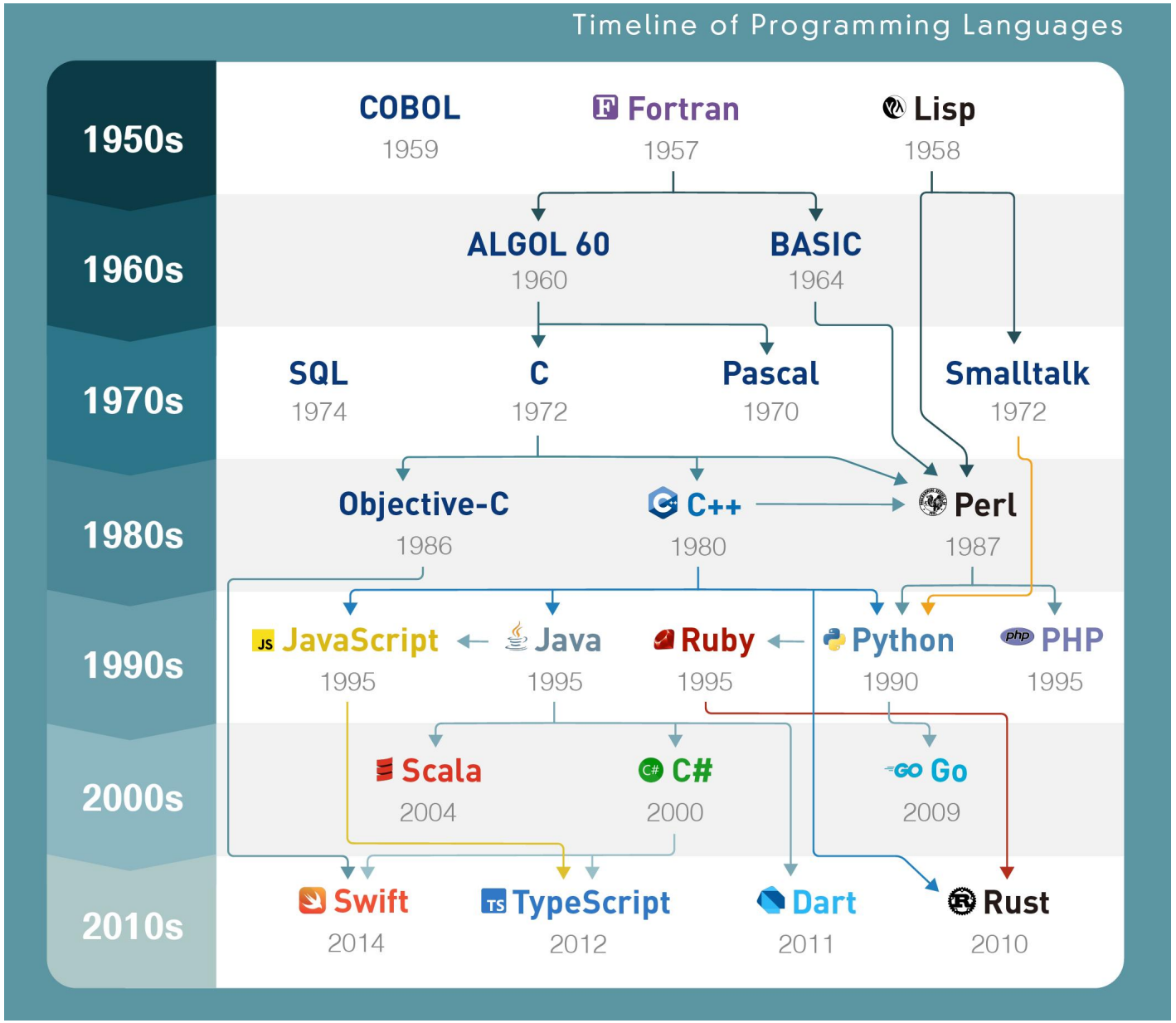
Chomsy语言体系

类型	文法名称	自动机模型	生成式形式	语言示例
0 型	递归枚举	图灵机	无限制	
1 型	上下文敏感	Linear bounded TM	左侧可以多个符号 $\alpha S \rightarrow \beta$	$a^n b^n c^n$
2 型	上下文无关	下推自动机	左侧仅一个符号$S \rightarrow \beta$	$a^n b^n$
3 型	正则	有穷自动机	右侧全部为终结符$S \rightarrow$<a><b>	a^n



计算机语言分类

- 按照执行方式分类
  - 命令式
  - 函数式
  - 声明式
- 按照运行过程分类
  - 编译性
  - 翻译性
- 按照数据类型分类
  - 静态类型
  - 动态类型



命令式编程

- 命令式编程语言是接近计算机硬件实现的语言设计
  - 很容易翻译成计算机指令
  - 执行顺序很关键
  - 代表语言：Fortran、Basic、C
  - 主要组成：
    - 赋值语句：内存存取、运算、以及更复杂的运算赋值形式
    - 循环语句：for、while等
    - 条件/非条件跳转语句

```
int fact(int n) {
    int sofar = 1;
    while (n>0) sofar *= n--;
    return sofar;
}
```

函数式编程

- 将电脑运算视为函数运算，避免或减少副作用.
  - 函数是一类“公民”，可以作为参数传递
  - 副作用：除了返回值以外对主调用函数产生的影响。
  - 主要特点：单赋值、多用递归实现
  - 代表语言：Haskell、ML、Lisp等
  - 基于λ演算（λ-calculus）的思想

```
(defun factorial (n)
  (if (= n 0) 1
      (* n (factorial (- n 1))) ) )

(loop for i from 0 to 16
   do (format t "~D! = ~D~%" i (factorial i)) )
```

声明式编程

- 描述目标的性质，而非流程，避免或减少副作用
- 包括以下子类：
  - 函数式编程：如Haskell、Lisp
  - 领域专属语言：如SQL、XML
  - 逻辑式编程：如Prolog
  - 约束式编程：如SMT-Lib

```
(set-logic QF_LIA)
(declare-const result Int)
(define-fun-rec
  fac ((x Int)) Int (
    ite (<= x 1)  1
    (* x (fac (- x 1)))
  )
)
(assert (= (fac 4) result))
(check-sat)
(get-value(result))
```

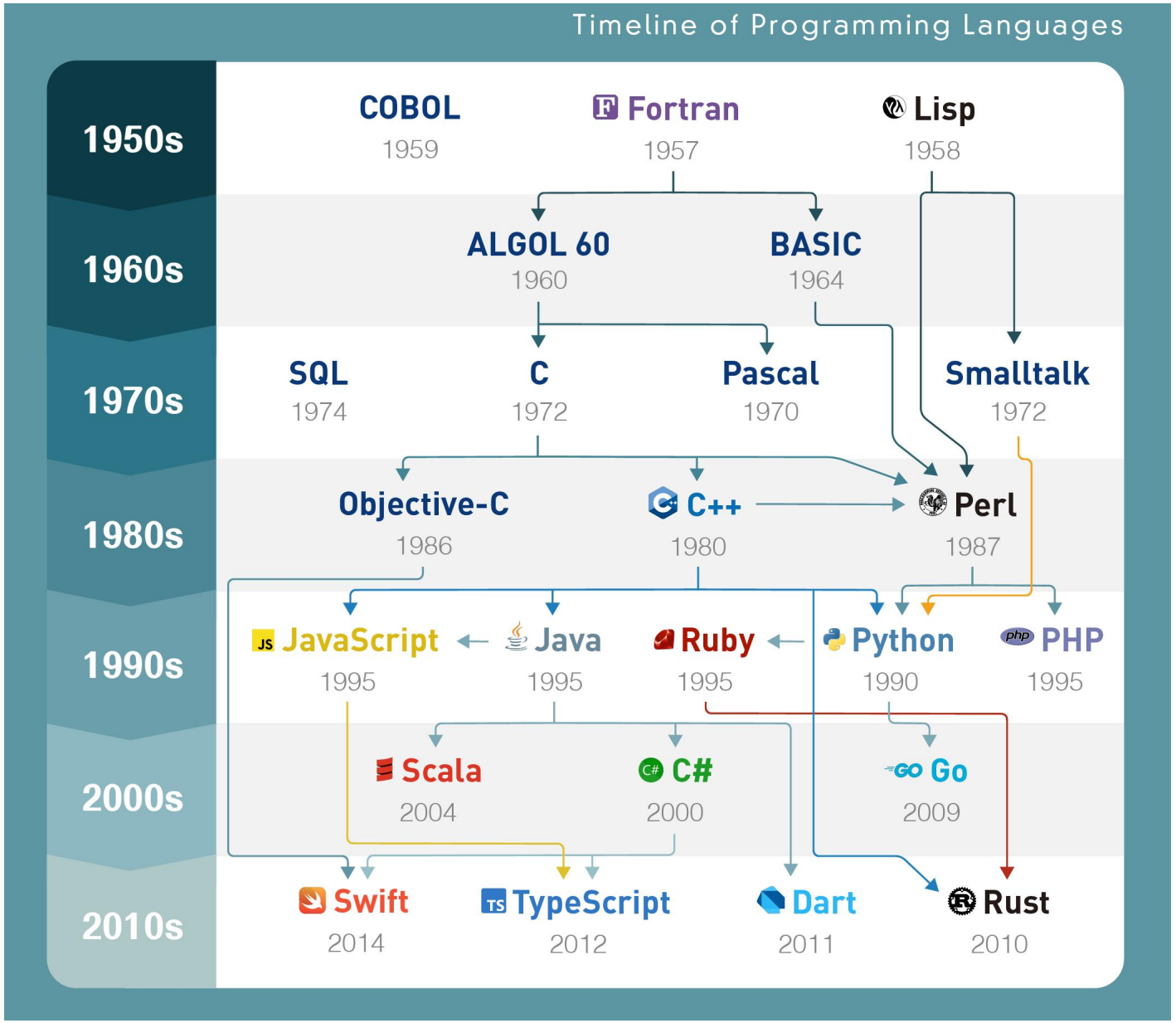
SMT-Lib代码

```
fact(X,1) :-
    X =:= 1.
fact(X,Fact) :-
    X > 1,
    NewX is X - 1,
    fact(NewX,NF),
    Fact is X * NF.
```

Prolog代码

计算机语言分类

- 按照执行方式分类
  - 命令式
  - 函数式
  - 声明式
- 按照运行过程分类
  - 编译性
  - 翻译性
- 按照数据类型分类
  - 静态类型
  - 动态类型



语言汇总

- C/Rust
  - Source -> llvm -> exe
- Java
  - Source -> java/Dalvik bytecode -> JVM / Graal VM
- Python/JavaScript
  - Source -> engine
- TypeScript
  - Source -> compile -> JavaScript / exe-level bytecode

编译执行和解释执行

- 编译执行
  - 通过编译器将源代码翻译为可以直接运行的机器码
  - 如C/C++、Rust等语言
- 解释执行
  - 在目标机器上将源代码翻译成计算机指令并执行
  - 如javascript、python等各种脚本语言
  - 平台无关，但存在一定的运行时翻译开销
- 混合模式
  - 如Java等语言通过编译器将源代码翻译为中间代码
  - 平台无关，省略代码解析开销
    - 例如 Android App => Dalvik bytecode
  - 优化：JIT+AOT

Dalvik bytecode

- move vA, vB
- return vAA
- const/4 vA, #+B
- check-cast vAA, type@BBBB
- instance-of vA, vB, type@CCCC
- array-length vA, vB
- new-instance vAA, type@BBBB
- new-array vA, vB, type@CCCC
- goto +AA

switch vAA, +BBBBBBBB

cmp*kind* vAA, vBB, vCC

if-*test* vA, vB, +CCCC

arrayop vAA, vBB, vCC

instanceop vA, vB, field@CCCC

invoke-*kind* {vC, vD, vE, vF, vG}, meth@BBBB

unop vA, vB

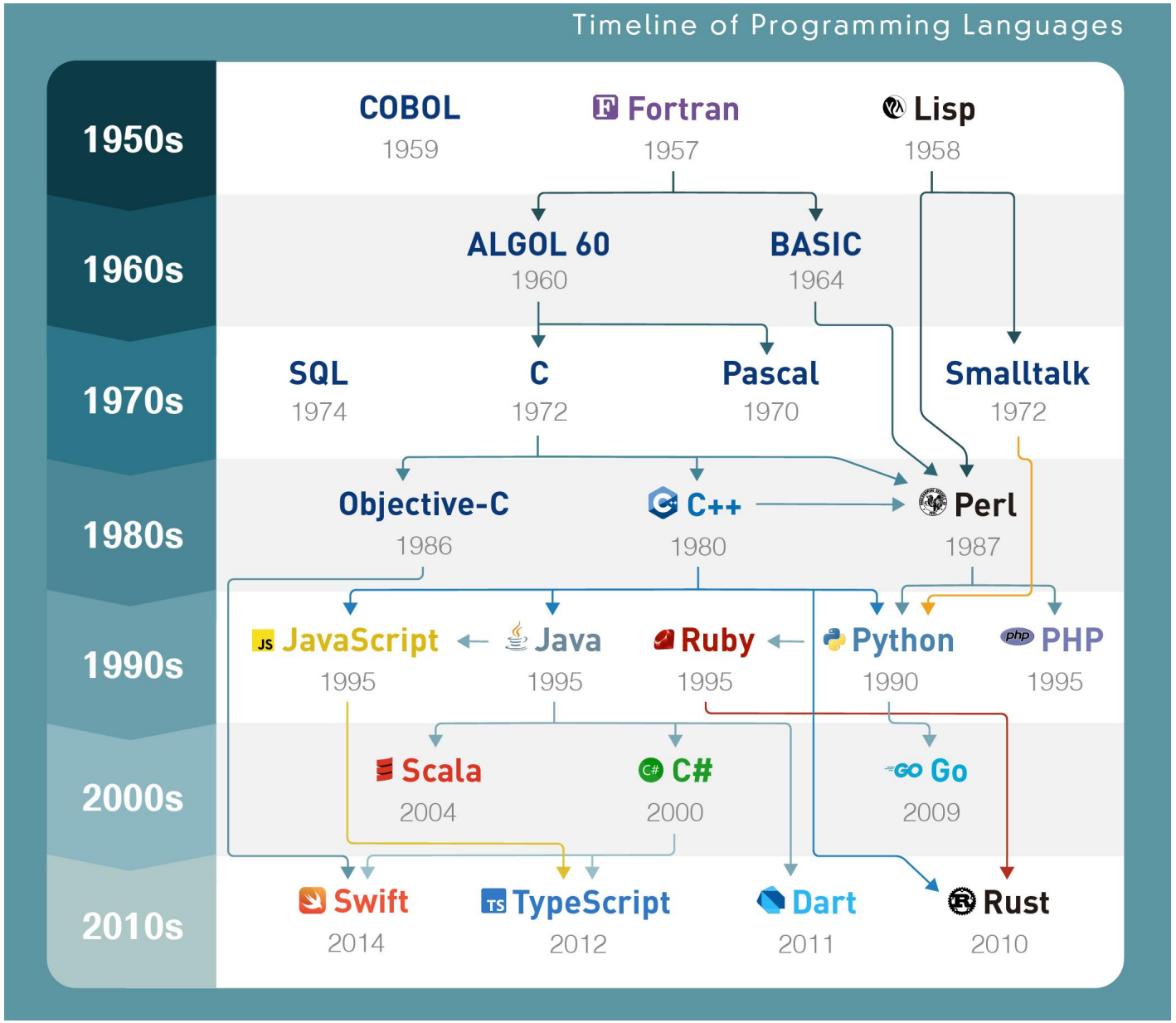
binop vAA, vBB, vCC

解释性语言优化：JIT和AOT

- JIT（just-in-time compilation）： 运行时编译
  - 将一部分需要频繁运行的代码直接编译为机器码。
  - 降低解释执行的运行时开销
  - 在javascript、JVM等解释执行引擎中广泛使用。
- AOT（Ahead-of-Timecompilation）
  - 将代码提前编译为机器码（传统C/C++等编译执行语言）
  - 将Java代码直接编译为机器码
    - 安卓智能手机安装app直接对其进行编译

计算机语言分类

- 按照执行方式分类
  - 命令式
  - 函数式
  - 声明式
- 按照运行过程分类
  - 编译性
  - 翻译性
- 按照数据类型分类
  - 静态类型
  - 动态类型



静态和动态语言

- 静态类型
  - 编译时确定数据类型
  - 通过类型检查避免运行时错误
  - 如C/C++、Java等语言
  - 一般写程序时需要为变量指定具体的类型，但不绝对
- 动态类型
  - 运行时确定数据类型
  - 如Python等各种语言
  - 一般写程序时不用为变量指定具体的类型